

COMP 2522 — Lab 6: Practicing Lambdas with a Hockey Team

Purpose

In this lab use **Java’s built-in functional interfaces** with lambda expressions. The focus is on writing lambdas and applying them in simple loop-based tasks.

You will **not** be learning new domain concepts — the “Hockey Team” model is deliberately tiny so the new idea you practice is *only* lambdas.

Learning Outcomes

By the end of this lab, you will be able to:

- Write and use lambda expressions with:
 - Predicate<T>
 - Function<T,R>
 - Consumer<T>
 - Supplier<T>
 - UnaryOperator<T>
 - Comparator<T>
- Write and use one **custom functional interface**.
- Apply lambdas inside ordinary **loops** (not streams).
- Sort, filter, and transform data using lambdas.

Setup

1. Create a package:
2. ca.bcit.cst.comp2522.lambdas
3. Inside this package, create **three simple classes**:
 - HockeyPlayer — represents a player.
 - HockeyTeam — holds a team name and a list of players.
 - Main — your main driver class with public static void main.
4. Keep the model tiny:
 - HockeyPlayer must have **only four fields**:
 - name (String)

- position (String, "F", "D", or "G")
 - yearOfBirth (int)
 - goals (int)
 - HockeyTeam has:
 - name (String)
 - roster (List<HockeyPlayer>)
5. Provide a **small roster** of six sample players in your main method.

In the main() method of the Main class, write the following code:

1. Supplier

- Write a Supplier<HockeyPlayer> that creates a “call-up” player (a new HockeyPlayer instance).
- Add this new player to the team’s roster.

2. Predicate

- Write a Predicate<HockeyPlayer> that checks if a player is a **Forward**.
- Write another Predicate<HockeyPlayer> that checks if a player has **20 or more goals**.
- Use these predicates in a loop to print only forwards with 20+ goals.

3. Function

- Write a Function<HockeyPlayer, String> that maps a player to a label string, e.g.:
 - Alex Morgan — 21G

4. Consumer

- Write a Consumer<HockeyPlayer> that prints just the player’s name.
- Loop through the roster and apply it.

5. UnaryOperator

- Write a UnaryOperator<String> that converts a string to uppercase.
- Use it to print all player names in uppercase.

6. Comparator

- Write a Comparator<HockeyPlayer> (as a lambda) that sorts players by **goals descending**.

- Sort the roster and print the results.

7. Aggregation (loop-based)

- Using a plain loop, calculate and print the team's **total goals**.

8. Custom Functional Interface

- Define your own `@FunctionalInterface` called `EligibilityRule`. It should declare a method:
 - `boolean test(HockeyPlayer player, int minAge, int minGoals, int currentYear);`
- Implement it with a lambda that says a player is **eligible** if:
 - their age (derived from `yearOfBirth`) is at least `minAge`
 - and they have at least `minGoals`.
- Test it by printing all eligible players when `minAge = 20` and `minGoals = 15`.

Deliverables

1. Your complete set of classes in the package `ca.bcit.cst.comp2522.lambdas`.
2. A main method that demonstrates each of the tasks above.
3. Output that clearly shows each step working.

Here is what `main()` could look like:

```
public class Main {  
  
    private static HockeyTeam sampleTeam() {  
  
        final List<HockeyPlayer> ps;  
        ps = new ArrayList<>();  
  
        ps.add(new HockeyPlayer("Alex Morgan", "F", 2002, 21));  
        ps.add(new HockeyPlayer("Ben Carter", "D", 1999, 6));  
        ps.add(new HockeyPlayer("Casey Young", "F", 2004, 28));  
        ps.add(new HockeyPlayer("Drew Singh", "G", 2000, 0));  
        ps.add(new HockeyPlayer("Eva Chen", "D", 2001, 5));  
        return new HockeyTeam("BCIT Blizzards", ps);  
    }  
}
```

```
public static void main(final String[] args) {  
    final int currentYear = 2025;  
  
    final HockeyTeam team = sampleTeam();  
    final List<HockeyPlayer> roster = team.getRoster();  
  
    // 1) Supplier — create a call-up and add it  
    // TODO: Write a Supplier<HockeyPlayer> lambda here  
    // Supplier<HockeyPlayer> callUp = ...  
    // roster.add(callUp.get());  
  
    // 2) Predicate — forwards with 20+ goals  
    // TODO: Write Predicate<HockeyPlayer> lambdas: isForward, has20Plus  
    // Use them in a loop to print forwards with >=20 goals  
  
    // 3) Function — map player to a label string  
    // TODO: Write Function<HockeyPlayer,String> lambda  
  
    // 4) Consumer — print names  
    // TODO: Write Consumer<HockeyPlayer> lambda  
  
    // 5) UnaryOperator — uppercase names  
    // TODO: Write UnaryOperator<String> lambda  
  
    // 6) Comparator — sort by goals DESC (no chaining)  
    // TODO: Write Comparator<HockeyPlayer> lambda  
    // Collections.sort(roster, yourComparator);  
  
    // 7) Aggregation (loop) — total goals
```

```
// TODO: Use a loop to sum goals across the roster

// 8) Custom FI (EligibilityRule)
// TODO: Write an EligibilityRule lambda
// A player is eligible if age >= minAge AND goals >= minGoals
}
```

```
}
```